

Image Segmentation

1. Introduction

Image Segmentation is the process of partitioning an image into multiple segments or regions, each assigned a class label.

Unlike classification (which labels an entire image), segmentation performs **pixel-wise classification**, assigning a label to each pixel.

- **Goal:** To simplify or transform the image representation for easier analysis, especially at the pixel level.
-

2. Image Classification vs. Segmentation

Feature	Image Classification	Image Segmentation
Output	Single label for the whole image	Label for every pixel
Detail Level	Coarse (global)	Fine-grained (pixel-level)
Example	Label image as "cat"	Label each pixel as "cat", "dog", "bg", etc.

3. Types of Segmentation

1. Semantic Segmentation

Labels each pixel with a class (e.g., "cat", "road"), but **does not** distinguish between multiple objects of the same class.

2. Instance Segmentation

Labels each pixel and also **differentiates between different instances** of the same object class (e.g., Cat 1, Cat 2).

4. How Image Segmentation Works

The process involves deep neural networks, usually convolution-based, and follows this structure:

1. **Input Image:** Passed into a CNN for feature extraction.
2. **Encoder (Downsampling):** Reduces spatial dimensions while capturing abstract features.
3. **Decoder (Upsampling):** Reconstructs the feature maps back to the original resolution.

4. **Output:** A segmentation mask of the same size as input, with each pixel labeled.

5. Popular Architectures

1. Fully Convolutional Networks (FCNs)

- Replaces fully connected layers with convolutional layers.
- Uses **transposed convolutions** for upsampling.

2. U-Net

- Encoder-decoder structure with **skip connections**.
- Useful for medical imaging where spatial detail is crucial.

3. SegNet

- Similar to U-Net, but uses **pooling indices** for efficient upsampling.
- Computationally lightweight.

4. Mask R-CNN

- Extension of Faster R-CNN for **instance segmentation**.
 - Combines object detection with pixel-level mask prediction.
-

6. Applications of Image Segmentation

Domain	Use Case
Medical Imaging	Tumor/organ segmentation from MRI or CT scans
Autonomous Driving	Lane, pedestrian, vehicle detection
Satellite Imagery	Land cover classification (forest, water, urban)
Precision Agriculture	Crop health segmentation, disease detection
Augmented Reality	Real-time background removal

7. Evaluation Metrics

1. Intersection over Union (IoU)

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

2. Dice Coefficient

$$\text{Dice} = \frac{2 \times \text{Intersection}}{\text{Total Pixels in Prediction} + \text{Ground Truth}}$$

3. Pixel Accuracy

- Measures percentage of correctly classified pixels.

4. Mean Accuracy (Per Class)

- Averages accuracy for each class, useful when classes are imbalanced.
-

8. Best Practices for Building Segmentation Models

1. Model Selection

- Choose based on task complexity (e.g., U-Net for medical, Mask R-CNN for object instances).

2. Data Preprocessing

- Resize images, normalize pixel values, apply augmentation (flip, rotate).

3. Handling Class Imbalance

- Use loss functions like **Dice Loss** or **Focal Loss** to prioritize minority classes.

4. Post-Processing

- Apply **CRFs (Conditional Random Fields)** to smooth the final segmented output.

5. Model Optimization

- Use efficient batching, tune hyperparameters (learning rate, dropout, batch size), and monitor GPU usage.
-

Image Segmentation vs Object Detection (Detailed Notes)

1. Goal

Task	Goal
Object Detection	Identify and localize multiple objects in an image using bounding boxes.
Image Segmentation	Assign a class label to each pixel in the image (pixel-wise classification).

2. Output Format

- **Object Detection:**
 - Returns: Bounding box coordinates + Class label

- Example: A dog inside a rectangle with label "dog"
 - **Image Segmentation:**
 - Returns: A pixel-level mask (same size as input image), where each pixel is assigned a class label.
 - Types:
 - **Semantic Segmentation:** All objects of the same class share the same label.
 - **Instance Segmentation:** Each object instance is uniquely identified.
-

3. Working Mechanism

Object Detection

- **Process:**
 1. Divide the image or use feature maps.
 2. Predict class label and bounding box coordinates.
 3. Apply Non-Max Suppression to filter overlapping boxes.
 - **Popular Architectures:**
 - YOLO (You Only Look Once)
 - SSD (Single Shot Detector)
 - Faster R-CNN
-

Image Segmentation

- **Process:**
 1. Input image is passed through CNN to extract features.
 2. Downsampling (encoder) captures context.
 3. Upsampling (decoder) recovers spatial details.
 4. Output is a segmentation mask (pixel-wise class predictions).
 - **Popular Architectures:**
 - FCN (Fully Convolutional Networks)
 - U-Net
 - SegNet
 - Mask R-CNN
-

4. Applications

Application Area	Object Detection	Image Segmentation
Medical Imaging	Detect tumors or organs with bounding boxes	Segment tumors, organs, and tissues at pixel level
Autonomous Driving	Detect cars, pedestrians, traffic signs	Segment road, lanes, sidewalks
Satellite Imaging	Detect buildings or vehicles	Segment land types: forest, water, urban
Augmented Reality	Detect objects for overlay	Segment background and foreground for realism
Precision Agriculture	Detect animals or machinery	Segment crops by health level

5. Timeline of Object Detection Models

Year	Model	Description
2001	Viola-Jones	First real-time face detector using Haar-like features and cascades
2005	HOG + SVM	Feature-based detection; slow and computationally expensive
2008	DPM	Deformable Part Model; part-based representation, robust but slow
2014	R-CNN	Region proposal + CNN; very slow, high accuracy
2015	Fast R-CNN	End-to-end training, faster than R-CNN, still uses external proposals
2015	Faster R-CNN	Introduced Region Proposal Network (RPN) for faster object proposals
2016	YOLOv1	Unified architecture, real-time detection using single network pass
2016	SSD	Detects objects at multiple scales using feature maps
2018	YOLOv3	Improved YOLO with multi-scale detection and residual blocks
2018	RetinaNet	Focal loss for class imbalance, ResNet + FPN backbone
2020	EfficientDet	Compound scaling for speed and accuracy balance
2020	YOLOv5	Ultralytics release, optimized for speed and deployment
2022	YOLOv6	Optimized for edge devices, efficient speed-accuracy tradeoff
2022	YOLOv7	State-of-the-art performance, with DLA and Efficient Layer Aggregation
2023	YOLOv8	Modular design, cross-platform, Ultralytics Hub integration

6. Evaluation Metrics

For Image Segmentation:

1. Intersection over Union (IoU):

- Measures overlap between prediction and ground truth.
- $\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$

2. Dice Coefficient:

- A similarity metric.
- $\text{Dice} = 2 \times (\text{Intersection}) / (\text{Total Pixels in Prediction} + \text{Ground Truth})$

3. Pixel Accuracy:

- Percentage of correctly labeled pixels.

4. Mean Class Accuracy:

- Average accuracy per class, useful for imbalanced datasets.
-

7. Key Differences Summary

Feature	Object Detection	Image Segmentation
Output Type	Bounding Boxes	Pixel-wise Mask
Granularity	Coarse (object-level)	Fine (pixel-level)
Instance Awareness	Yes (in all models)	Only in Instance Segmentation
Speed (typical)	Faster (YOLO, SSD)	Slower due to dense prediction
Applications	Tracking, counting, event detection	Detailed understanding, scene segmentation

8. Extra Concepts

Classification vs Localization vs Detection

- **Classification:** What is in the image? (No location info)
 - **Localization:** What and where is the object? (One object, one bounding box)
 - **Detection:** What and where are multiple objects? (Bounding boxes for multiple objects)
-

9. Practical Considerations

While Building Segmentation Models:

- **Model Choice:** Use U-Net for biomedical, Mask R-CNN for instance-level.

- **Data Preprocessing:** Normalize, resize images, use augmentations.
 - **Class Imbalance:** Use Dice Loss, Focal Loss.
 - **Post-Processing:** Apply CRFs or morphological operations for cleaner masks.
 - **Optimization:** Tune hyperparameters, batch sizes, and use GPU acceleration.
-

✓ **Conclusion:**

- **Object Detection** is about "what is there and where?"
- **Image Segmentation** is about "what is there, where, and at pixel level?"

Together, they enable machines to interpret visual scenes with varying levels of granularity.

Evolution of Object Detection Models (2001–2023)

This timeline summarizes the key milestones in the evolution of object detection techniques from classical methods to modern deep learning-based approaches.

2001 – Viola-Jones Detector

- **Key Idea:** Haar-like features + AdaBoost + sliding window.
 - **Strengths:** Real-time face detection.
 - **Limitations:** Hand-crafted features, not generalizable to complex objects.
-

2005 – Histogram of Oriented Gradients (HOG) + SVM

- **Proposed by:** Dalal and Triggs
 - **Key Idea:** Uses edge orientation histograms as features, combined with Support Vector Machines (SVM).
 - **Application:** Pedestrian detection.
 - **Limitations:** Computationally expensive and inflexible.
-

2013 – R-CNN (Regions with CNN Features)

- **Proposed by:** Ross Girshick
- **Key Idea:** Combines region proposals with deep learning features.
- **Pipeline:**
 - Extract region proposals using Selective Search.

- Compute CNN features for each region.
 - Use SVM for classification and regression for bounding box adjustment.
 - **Limitation:** Slow due to multiple forward passes (≈ 2000 per image).
-

2015 – Fast R-CNN

- **Improved by:** Ross Girshick
 - **Key Improvements:**
 - Performs region proposals and classification in a single forward pass.
 - Uses **RoI Pooling** to extract fixed-size features.
 - **Speed:** Much faster than R-CNN.
 - **Limitation:** Still depends on Selective Search for proposals.
-

2015 – Faster R-CNN

- **Proposed by:** Shaoqing Ren et al.
 - **Innovation:** Introduced **Region Proposal Network (RPN)** to generate object proposals within the network.
 - **Advantages:** Fully end-to-end and real-time capable on GPUs.
-

2016 – YOLO (You Only Look Once) v1

- **Key Idea:** Single-stage detector that divides the image into a grid and predicts bounding boxes + class probabilities directly.
 - **Strength:** Extremely fast (real-time).
 - **Trade-off:** Less accurate for small objects or dense scenes.
-

2016 – SSD (Single Shot Multibox Detector)

- **Key Idea:** Predicts bounding boxes at multiple scales using feature maps from different layers.
 - **Advantage:** Balanced speed and accuracy.
 - **Use Case:** Mobile and embedded devices.
-

2017 – YOLO v2 (YOLO9000)

- **Enhancements:**
 - Better backbone (Darknet-19).

- Joint detection + classification using WordTree.
 - **Improved accuracy over YOLOv1.**
-

2018 – YOLO v3

- **Upgrades:**
 - Multi-scale predictions using Feature Pyramid Networks (FPN).
 - New backbone: Darknet-53.
 - Better detection of small objects.
 - **Trade-off:** Slightly slower but more accurate.
-

2018 – RetinaNet

- **Key Idea:** Combines one-stage detection with **Focal Loss** to address class imbalance.
 - **Performance:** Comparable to Faster R-CNN in accuracy but faster.
-

2018 – Mask R-CNN

- **Extension of:** Faster R-CNN
 - **Additional Output:** Pixel-wise segmentation mask along with class + bounding box.
 - **Use Case:** Instance segmentation.
-

2020 – EfficientDet

- **Backbone:** EfficientNet.
 - **Key Feature:** BiFPN (Bidirectional Feature Pyramid Network).
 - **Result:** State-of-the-art accuracy with high efficiency on edge devices.
-

2021 – YOLO v4 & v5

- **Community Developed:** (AlexeyAB, Ultralytics)
 - **YOLOv4:** CSPDarknet + PANet + Bag-of-Freebies.
 - **YOLOv5:** Lightweight, PyTorch-based, easy to train and deploy.
-

2022 – YOLOv6 & YOLOv7

- **YOLOv6:** Tailored for industrial applications (ByteDance).

- **YOLOv7:** Unified architecture for object detection and instance segmentation with even better performance.
-

2023 – YOLOv8

- **By:** Ultralytics
 - **New Features:**
 - Fully decoupled head (improves accuracy).
 - Supports tasks like segmentation, classification, and pose estimation.
 - Easier deployment with ONNX, TensorRT, etc.
 - **Use Case:** Production-ready, fast and modular.
-

Summary Table: Object Detection Milestones

Year	Model	Type	Key Feature	Speed	Accuracy
2001	Viola-Jones	Classic	Haar + AdaBoost	High	Low
2013	R-CNN	Two-Stage	Selective Search + CNN	Low	High
2015	Fast R-CNN	Two-Stage	RoI Pooling	Medium	High
2015	Faster R-CNN	Two-Stage	RPN (end-to-end)	Medium	Very High
2016	YOLO v1	One-Stage	Grid-based prediction	Very High	Medium
2016	SSD	One-Stage	Multi-scale detection	High	Good
2017	YOLO v2	One-Stage	Better backbone (Darknet-19)	High	Better
2018	YOLO v3	One-Stage	FPN + Darknet-53	High	High
2018	RetinaNet	One-Stage	Focal Loss	High	Very High
2018	Mask R-CNN	Two-Stage	Adds segmentation mask	Medium	Very High
2020	EfficientDet	One-Stage	BiFPN + EfficientNet	High	Very High
2021	YOLO v4/v5	One-Stage	Modular & optimized for speed	Very High	Very High
2022	YOLO v6/v7	One-Stage	Unified design for multiple tasks	Very High	SOTA
2023	YOLO v8	One-Stage	Decoupled head + Task versatility	Very High	SOTA

Object Detection - Detailed Notes

Introduction

Object Detection is a computer vision technique that **identifies** and **localizes** multiple objects within an image using **bounding boxes** and **class labels**.

- **Purpose:** Classify objects and provide their spatial location (bounding box).
 - **Difference from Classification:** Classification tells *what* is in the image. Object Detection tells *what* and *where*.
 - **Applications:** Self-driving cars, facial recognition, video surveillance, medical imaging.
-

Working of Image Classification

1. **Input Image:** The image is resized (e.g., 224×224) for compatibility with CNNs like VGG/ResNet.
 2. **Feature Extraction:**
 - CNN layers extract features like edges, shapes, and textures.
 - Pooling layers reduce spatial dimensions.
 3. **Flattening:** Final feature maps are converted into a 1D vector.
 4. **Fully Connected Layers:** Act as decision-making layers.
 5. **Softmax Layer:** Outputs class probabilities. The highest probability class is selected as the label.
-

Working of Object Detection

1. **Input Image:** Passed into an object detection model (e.g., YOLO, SSD, Faster R-CNN).
 2. **Feature Extraction (Backbone CNN):** Extracts feature maps from the image.
 3. **Region Proposal or Grid Prediction:**
 - **RPN:** Proposes likely regions (Faster R-CNN).
 - **Grid:** Divides image into grids (YOLO).
 4. **Bounding Box Prediction:**
 - Predicts (x, y, width, height)
 - Outputs **confidence score** for object presence.
 5. **Class Label Prediction:** For each box, predicts object class (dog, cat, etc.)
 6. **Post-processing (NMS):** Removes overlapping boxes; keeps only highest confidence ones.
 7. **Output:** Final list of objects with bounding boxes, class labels, and confidence scores.
-

Bounding Box Representation

Bounding boxes localize objects using geometric coordinates.

- **(x, y):** Top-left corner coordinates.

- **Width (w), Height (h):** Size of the box.
- Alternative: **Center (cx, cy), width, height**

Coordinate Systems

- **Absolute:** (x, y, w, h) are pixel values.
 - **Normalized:** Values scaled between 0 and 1 relative to image size.
-

Axis-Aligned Bounding Boxes (AABB)

AABB is a rectangle aligned to the image's horizontal and vertical axes.

Characteristics:

- **Orientation:** Not rotated, always axis-aligned.
- **Parameters:** Same as regular bounding boxes: (x, y, w, h) or (cx, cy, w, h)

Advantages:

- Simple and fast to compute
- Widely used in object detection models

Limitations:

- Not precise for rotated or angled objects
- Includes extra background area

COCO vs YOLO Bounding Box Formats & Sliding Window Technique

COCO Format (Common Objects in Context)

COCO is a large-scale object detection dataset with a widely used annotation format. It stores bounding box information using **absolute pixel values**.

Bounding Box Format (COCO):

(x_min, y_min, width, height)

Parameter	Description
x_min	X-coordinate of the top-left corner
y_min	Y-coordinate of the top-left corner
width	Width of the bounding box (in pixels)
height	Height of the bounding box (in pixels)

- All values are **absolute**, not relative.
- The top-left corner acts as the reference point.
- The box is extended **rightwards and downwards**.

YOLO Format (You Only Look Once)

YOLO uses a **normalized format**, where all coordinates and dimensions are relative to image size. This makes the model scalable to various image resolutions.

Bounding Box Format (YOLO):

(x_center, y_center, width, height)

Parameter	Description
x_center	Normalized x-coordinate of the bounding box center (0–1)
y_center	Normalized y-coordinate of the bounding box center (0–1)
width	Normalized width of the bounding box (0–1)
height	Normalized height of the bounding box (0–1)

- **Normalized** = divide by image width/height.
- Center of the box is used as reference.
- Scales well across various image sizes.

COCO vs YOLO — Summary Table

Feature	COCO Format	YOLO Format
Coordinates	Absolute (pixels)	Normalized (0–1)
Reference Point	Top-left corner	Center of bounding box
Parameters	x_min, y_min, width, height	x_center, y_center, width, height
Flexibility	Tied to image size	Scalable to any image size

Feature	COCO Format	YOLO Format
Usage	COCO Dataset, Faster R-CNN, etc	YOLO, Real-time applications

Sliding Window Technique in Object Detection

Definition:

The **sliding window technique** involves scanning an image by moving a fixed-size window across it, detecting objects within each window individually.

How It Works:

1. Window Size:

- A fixed-size window (e.g., 32x32 pixels) is chosen.

2. Movement (Stride):

- The window moves left to right, top to bottom with a certain step size.

3. Multi-Scale:

- To detect objects of different sizes, either the image is resized or the window is scaled.

4. Classification:

- Each window is passed through a classifier (e.g., CNN) to check for object presence.
-

Example Use Case:

- Used in **traditional object detection**, such as HOG + SVM or early CNN-based methods.
 - Multiple windows are generated and evaluated for object presence.
-

Advantages:

- Simple and intuitive.
 - Useful when object positions are roughly known or in small datasets.
-

Disadvantages:

Problem	Explanation
High Computational Cost	Many windows to evaluate, especially at multiple scales.
Redundant Processing	Most windows contain no object.
Fixed Window Size	Poor at detecting objects of varying sizes/aspect ratios.
Not Real-Time Friendly	Too slow for modern real-time applications.



Modern Solutions:

Technique	How It Helps
Region Proposal Networks (RPNs)	Generate fewer, smarter regions (used in Faster R-CNN).
Anchor Boxes (YOLO, SSD)	Predefined boxes for various sizes and shapes.
Fully Convolutional Networks (FCNs)	Operate on the whole image at once.
Feature Pyramids	Use multi-scale features for detecting objects at various sizes efficiently.

✓ Sliding Window is foundational but now largely replaced by more **efficient and scalable** techniques like RPNs and anchor boxes in modern object detection.

In []: